Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №6
Декораторы, контекстные менеджеры и расширенные возможности Python

1. Цель работы

Изучить и получить практические навыки использования расширенных возможностей языка Python: аннотаций типов, dataclass, перечислений Enum, декораторов функций и методов, декораторов с параметрами, контекстных менеджеров, протоколов Protocol, а также механизмов интроспекции объектов.

2. Постановка задачи

Разработать консольное приложение для управления объектами предметной области «Гостиница» (Вариант 5). Объекты предметной области: Номер, Бронирование, Гость. Статусы номера (Enum): свободен, занят, забронирован, на обслуживании. Дополнительные операции: бронирование номера, расчёт стоимости проживания.

Технические требования: приложение должно содержать несколько типов объектов, использовать dataclass, Enum, аннотации типов; реализовать минимум три собственных декоратора (журналирование, измерение времени, декоратор с параметрами), собственный контекстный менеджер, минимум один Protocol, механизмы интроспекции объектов, консольное меню, обработку исключений (включая собственное кастомное исключение) и формирование итоговой статистики.

3. Краткие теоретические сведения

В Python dataclass позволяет автоматически генерировать методы инициализации, сравнения и представления для классов, предназначенных для хранения данных. Перечисления Enum обеспечивают безопасную работу с фиксированным набором констант. Декораторы представляют собой функции, принимающие другую функцию и расширяющие её поведение без изменения исходного кода. Контекстные менеджеры (реализуемые через __enter__/__exit__ или @contextmanager) гарантируют выполнение определённых действий при входе и выходе из блока кода. Protocol обеспечивает структурную типизацию, а

интроспекция позволяет исследовать структуру объектов во время выполнения программы.

4. Исходный код программы

Файл: exceptions.py — собственные исключения

```
class HotelError(Exception):
    """Базовое исключение для предметной области."""
    pass

class RoomNotAvailableError(HotelError):
    """Номер недоступен для бронирования."""
    pass

class EntityNotFoundError(HotelError):
    """Объект не найден в системе."""
    pass
```

Файл: protocols.py — протокол Reportable

```
from typing import Protocol

# Протокол
class Reportable(Protocol):
    def get_report_data(self) -> str:
        ...
```

Файл: models.py — dataclass, Enum, протокол

```
from dataclasses import dataclass
from enum import Enum
from typing import Optional
from exceptions import RoomNotAvailableError
from protocols import Reportable

# Перечисление
class RoomStatus(Enum):
    FREE = "свободен"
    OCCUPIED = "занят"
    BOOKED = "забронирован"
    MAINTENANCE = "на обслуживании"

# Базовый класс
@dataclass
class BaseEntity:
    id: int

# Специализированный класс 1: Гость
@dataclass
class Guest(BaseEntity):
    name: str
    passport: str

# Специализированный класс 2: Номер
@dataclass
class Room(BaseEntity):
    number: str
    price_per_night: float
    status: RoomStatus = RoomStatus.FREE

    # Реализация протокола Reportable (структурная типизация)
    def get_report_data(self) -> str:
        return f"Номер {self.number}, Цена: {self.price_per_night} руб., Статус:
{self.status.value}"

    # Дополнительная операция (расчёт стоимости проживания)
    def calculate_cost(self, days: int) -> float:
        if days <= 0:
            raise ValueError("Количество дней должно быть больше 0")
```

```
        return self.price_per_night * days

# Специализированный класс 3: Бронирование
@dataclass
class Booking(BaseEntity):
    guest_id: int
    room_id: int
    days: int
    total_cost: float

    def get_report_data(self) -> str:
        return f"Бронирование ID {self.id}: Гость {self.guest_id}, Номер
{self.room_id}, {self.days} дн., Стоимость: {self.total_cost} руб."
```

Файл: decorators.py — три собственных декоратора

```
import time
from functools import wraps
from exceptions import RoomNotAvailableError

# Декоратор №1 — журналирование
def log_call(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"[LOG] Вызван метод {func.__name__}()")
        result = func(*args, **kwargs)
        print(f"[LOG] Результат: {result}")
        return result
    return wrapper

# Декоратор №2 — измерение времени
def measure_time(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()
        print(f"[PERFORMANCE] {func.__name__}: {end - start:.5f} сек")
        return result
    return wrapper

# Декоратор №3 — с параметрами
def require_status(required_status):
    def decorator(func):
        @wraps(func)
        def wrapper(self, room, *args, **kwargs):
            if room.status != required_status:
                raise RoomNotAvailableError(
                    f"Номер {room.number} должен быть в статусе
'{required_status.value}', "
                    f"а сейчас он '{room.status.value}'"
                )
            return func(self, room, *args, **kwargs)
        return wrapper
    return decorator
```

Файл: context.py — контекстный менеджер

```
from contextlib import contextmanager

# Собственный контекстный менеджер
@contextmanager
def hotel_operation(operation_name: str):
    print(f"\n--- Начало операции: {operation_name} ---")
    try:
        yield
    except Exception as e:
        print(f"!!! Ошибка в операции '{operation_name}': {e} !!!")
        raise
    finally:
        print(f"--- Завершение операции: {operation_name} ---\n")
```

Файл: services.py — сервисный слой, статистика, интроспекция

```
import inspect
from typing import List, Optional
from models import Room, Guest, Booking, RoomStatus
from exceptions import EntityNotFoundError
# ДОБАВЛЕН ИМПОРТ ДЕКОРАТОРОВ
from decorators import log_call, measure_time, require_status
from context import hotel_operation

class HotelService:
    def __init__(self):
        self.rooms: List[Room] = []
        self.guests: List[Guest] = []
        self.bookings: List[Booking] = []

    def add_room(self, room: Room) -> None:
        self.rooms.append(room)

    def add_guest(self, guest: Guest) -> None:
        self.guests.append(guest)

    def find_room(self, room_id: int) -> Room:
        for room in self.rooms:
            if room.id == room_id:
                return room
        raise EntityNotFoundError(f"Номер с ID {room_id} не найден")

    def find_guest(self, guest_id: int) -> Guest:
        for guest in self.guests:
            if guest.id == guest_id:
                return guest
        raise EntityNotFoundError(f"Гость с ID {guest_id} не найден")

    # ИСПРАВЛЕНО: используем декораторы логов и времени, а контекстный менеджер
- внутри через with
    @log_call
    @measure_time
    def book_room(self, room_id: int, guest_id: int, days: int) -> Booking:
        room = self.find_room(room_id)
        guest = self.find_guest(guest_id)

        # ИСПРАВЛЕНО: контекстный менеджер вызывается через with
        with hotel_operation("Бронирование номера"):
            self._process_booking(room, guest, days)
            cost = room.calculate_cost(days)
            booking = Booking(
                id=len(self.bookings) + 1,
                guest_id=guest.id,
                room_id=room.id,
                days=days,
                total_cost=cost
            )
            self.bookings.append(booking)
            return booking

    # Декоратор с параметрами
    @require_status(RoomStatus.FREE)
    def _process_booking(self, room: Room, guest: Guest, days: int):
        room.status = RoomStatus.BOOKED
        print(f"Успешно: Гость '{guest.name}' забронировал номер {room.number}
на {days} дн.")

    # Статистика
    def get_statistics(self) -> dict:
        total_rooms = len(self.rooms)
        free_rooms = sum(1 for r in self.rooms if r.status == RoomStatus.FREE)
        booked_rooms = sum(1 for r in self.rooms if r.status ==
RoomStatus.BOOKED)

        prices = [r.price_per_night for r in self.rooms]
        min_price = min(prices) if prices else 0
        max_price = max(prices) if prices else 0
```

```
        return {
            "Всего номеров": total_rooms,
            "Свободных": free_rooms,
            "Забронировано": booked_rooms,
            "Мин. цена": min_price,
            "Макс. цена": max_price
        }

    # Интроспекция
    def inspect_object(self, obj: object) -> None:
        print("\n=== ИНФОРМАЦИЯ ОБ ОБЪЕКТЕ (ИНТРОСПЕКЦИЯ) ===")
        print(f"Тип объекта: {type(obj).__name__}")
        print(f"Является ли экземпляром Room: {isinstance(obj, Room)}")
        print(f"Является ли подклассом BaseEntity: {issubclass(type(obj), Room)
or issubclass(type(obj), Guest)}")
        print(f"Есть ли атрибут 'status': {hasattr(obj, 'status')}")
        if hasattr(obj, 'status'):
            print(f"Значение атрибута 'status': {getattr(obj, 'status')}")

        print("Методы объекта:")
        for name, method in inspect.getmembers(obj, predicate=inspect.ismethod):
            if not name.startswith('__'):
                print(f"  - {name}{inspect.signature(method)}")
        print("==========================================\n")
```

5. Результаты тестирования программы

Тест 1. Запуск программы и вывод списка номеров и гостей, предзаполненных для тестирования. Проверка корректного отображения статусов (рисунок 6.1).

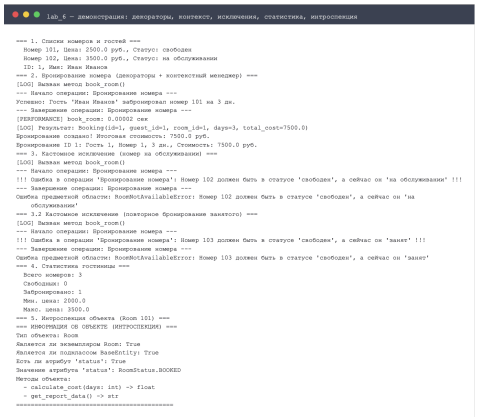


Рисунок 6.1 – Вывод списка номеров (верхняя часть демонстрации)

Тест 2. Выполнение дополнительной операции — бронирование номера. Демонстрация работы декораторов @log_call, @measure_time, контекстного менеджера hotel_operation и декоратора с параметрами @require_status.

Тест 3. Проверка работы кастомного исключения RoomNotAvailableError при попытке забронировать номер, который уже занят или находится на обслуживании. Тест 4. Формирование итоговой статистики по номерам гостиницы. Тест 5. Демонстрация механизмов интроспекции объекта: вывод типа объекта, проверка принадлежности к классу, наличие атрибутов и список методов с сигнатурами. Все сценарии показаны на рисунке 6.1.

Автоматические тесты модуля (pytest) — 17 проверок моделей, декораторов, контекстного менеджера, бронирования, статистики и интроспекции; все прошли успешно (рисунок 6.2).

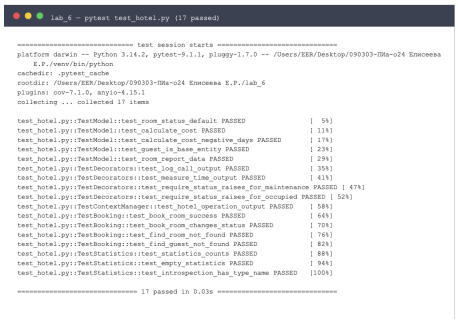


Рисунок 6.2 – Результаты pytest (17 passed)

6. Вывод

В процессе выполнения лабораторной работы были успешно освоены расширенные возможности языка Python. На практике изучено применение декораторов для расширения функциональности методов (логирование, замер времени, проверка предусловий), а также создание собственных контекстных менеджеров для безопасного управления ресурсами. Закреплены навыки использования dataclass для создания чистых моделей данных, Enum для

строгой типизации состояний и Protocol для реализации структурной типизации. Отработаны механизмы интроспекции объектов и создание кастомных исключений для повышения отказоустойчивости приложения.